Serial reception and HDLC timing

Capture bits on real edges, assemble complete frames, and place pattern outputs in the required cycle.

Kapil Tripathi | Verilog and digital design | 27 pages

CONTENTS / click a topic to jump

Reading days group the explanations, including later revisits. Tracker day labels and original dated submission evidence remain unchanged.

Use the linked contents or PDF bookmarks. Page numbers match the PDF viewer. Source questions and technical evidence remain beside their explanations.

Questions answered

1. Start from the clock sequence

The base receiver detects a start bit, waits for eight data bits, and validates the stop bit. The datapath version then preserves those eight sampled bits so that `out_byte` is valid when `done` is asserted. HDLBits explicitly says the serial stream is least-significant-bit first and is shifted in one bit at a time. [1][2]

Problem	What changes	Timing responsibility
Fsm_serial	Control only	State remembers start, 8 data clocks, stop, and framing errors.
Fsm_serialdata	Add byte datapath	A counter or shift register records exactly one data bit per clock.
Fsm_serialdp	Add parity bit and checker	The parity accumulator advances on the same serial clock sequence. [3]

The attempted loop

```
for (i = 1; i < 8; i = i + 1) begin
    count <= count + 4'd1;
    data_out[i] <= in;
end
```

> **Why the loop cannot wait**
>
> The loop variable `i` changes during one execution of the process. No `@(posedge clk)`, delay, or state transition appears inside the loop, so nothing advances simulation time and nothing waits for another clock edge.

What synthesis sees

AMD Vivado documents `for` and `repeat` as supported ways to describe repetitive or bit-slice structures inside an `always` block. [4] For constant loop bounds, the hardware meaning is a fixed amount of replicated logic, not a hidden clock sequencer.

2. One rising edge, seven loop iterations

Assume the pre-edge values are `count = 3` and `in = 1`. The clock event wakes the `always @(posedge clk)` process once. The process then executes all seven iterations before the next clock event can occur.

Iteration	RHS read for count	Scheduled bit write
i = 1	old count 3 -> 4	data_out[1] <= 1
i = 2	old count 3 -> 4	data_out[2] <= 1
i = 3	old count 3 -> 4	data_out[3] <= 1
...	same old count	same sampled input
i = 7	old count 3 -> 4	data_out[7] <= 1

Why nonblocking assignment matters

- AMD states that a nonblocking assignment evaluates its expression when the statement executes, while the variable changes later. [5]

- Therefore each `count <= count + 1` evaluates from the same pre-update `count` in this process activation.

- All seven scheduled values are identical here. The observable result after the edge is `count = 4`, not 10.

- The seven different bit-select targets are all scheduled, so `data_out[7:1]` becomes seven copies of the one sampled input value.

> **How the loop runs**
>
> Nonblocking assignment is not a mechanism for repeating work on later clocks. It separates evaluation from update within the current simulation time slot. Clock-to-clock behavior still requires stored state that changes once per clock.

IEEE 1800-2023 is the active SystemVerilog language standard defining the language's syntax and semantics, including RTL and testbench behavior. [6] AMD UG901 supplies the synthesis-oriented interpretation used here. [4][5]

3. Spatial repetition is not temporal repetition

Question	Spatial repetition	Temporal repetition
What repeats?	Hardware operations or bit-slice assignments	A registered action across clock edges
What advances it?	Loop variable during one process activation	Counter, FSM state, shift register, or handshake
How many input samples?	One sample for this clock event	One sample on each selected future clock
Typical RTL	for loop over vector bits	if (enable) count <= count + 1
UART byte capture	Wrong abstraction for consuming the stream	Exactly the required abstraction

A useful hardware picture

SPATIAL - same cycle

```
a[0] & b[0] -> out[0]
a[1] & b[1] -> out[1]
a[2] & b[2] -> out[2]
...
a[7] & b[7] -> out[7]
```

TEMPORAL - different cycles

```
edge 1: in -> data_out[0]
edge 2: in -> data_out[1]
edge 3: in -> data_out[2]
...
edge 8: in -> data_out[7]
```

Fast diagnostic question

Ask: 'What register tells the circuit which byte bit this clock belongs to?' If the answer is only the loop variable, the design has no clock-to-clock memory for that position.

4. Local simulation evidence

A small self-checking Icarus Verilog simulation was generated for this document. The first DUT contains the original loop. The second DUT uses one counter-controlled write per edge. The testbench fails immediately if either result differs from the expected clock semantics.

```
BAD one edge: count=1 data_out=11111110
GOOD eight edges: count=0 data_out=10100110
PASS: spatial loop and temporal counter behaviors verified
```

Interpreting the result

Observed result	Meaning
BAD count = 1	Seven identical nonblocking increment values were computed from old count = 0.
BAD data_out = 11111110	Bits 1 through 7 copied the single input value 1; bit 0 was untouched.
GOOD count = 0	Eight real rising edges occurred and the 3-bit position counter wrapped after bit 7.
GOOD data_out = 10100110	The LSB-first stream was reconstructed into the intended parallel byte.

> **What was checked**
>
> This test checks the loop and counter examples shown here. The complete original serial-receiver submission is not saved locally, so this is not a rerun of that submission.

5. Correct RTL capture patterns

Pattern A - indexed write with a registered bit counter

```
always @(posedge clk) begin
    if (reset) begin
        count    <= 3'd0;
        data_out <= 8'd0;
    end else if (state == DATA) begin
        data_out[count] <= in;

        if (count == 3'd7)
            count <= 3'd0;
        else
            count <= count + 3'd1;
    end else begin
        count <= 3'd0;
    end
end
```

At each data-state edge, the old registered `count` selects one destination bit. The nonblocking update then prepares the next count for the next edge. This is precisely the old-value behavior the design needs.

Pattern B - shift register

```
always @(posedge clk) begin
    if (reset)
        shift_reg <= 8'd0;
    else if (state == DATA)
        shift_reg <= {in, shift_reg[7:1]};
end
```

Because HDLBits transmits the least-significant bit first, shifting right and inserting the newest bit at bit 7 reconstructs the final byte after eight data clocks. [2] Either indexed capture or shifting is valid; use one consistent convention and verify it with a known asymmetric byte.

> **Off-by-one checkpoint**
>
> Decide whether `count` names the bit being captured now or the number already captured. Then keep state transitions, final-bit capture, stop-bit validation, and `done` aligned to that definition.

6. Eight-clock trace

Use the test byte `8'b10100110`. Since the protocol is LSB-first, the serial samples are 0, 1, 1, 0, 0, 1, 0, 1. The table shows values after each edge.

Edge	Old count	in	Write	Count after	data_out after
1	0	0	bit 0	1	xxxxxxx0
2	1	1	bit 1	2	xxxxxx10
3	2	1	bit 2	3	xxxxx110
4	3	0	bit 3	4	xxxx0110
5	4	0	bit 4	5	xxx00110
6	5	1	bit 5	6	xx100110
7	6	0	bit 6	7	x0100110
8	7	1	bit 7	0	10100110

Why the last row is safe

- The left-hand bit-select uses the old count value 7 for this edge.
- The count update to 0 occurs later in the nonblocking update phase.
- The completed byte can be consumed during the stop/done phase after the eighth data bit has already been stored.

> **Points to remember**
>
> A counter does not merely count; it is the circuit's memory of bit position. In this receiver, that position selects both what the current input means and where it must be stored.

7. Link the experience to the problem series

HDLBits problem	Experience link	Revision prompt
Fsm_serial [1]	State is time: start -> 8 data clocks -> stop/error.	Can the FSM recover if the stop bit is missing?
Fsm_serialdata [2]	Main lesson: one input sample and one byte-position update per data edge.	Can you trace an asymmetric byte LSB-first?
Fsm_serialdp [3]	Extend the same temporal chain by one parity-bit clock; reset parity at the frame boundary.	Which edge resets parity and which edge tests it?
Fsm_hdlc	Related Day 1 lesson: registered history and output phase matter more than procedural-looking code.	Does the output describe this input now or a remembered sequence?

The three linked exercises

The tracker links entries 2, 4 and 6 to this PDF. They share the same capture sequence: a start bit, eight data edges, and a stop check. Entry 6 adds a parity edge before the stop bit. Pages 11-12 show that timing.

Progress and local checks

The tracker records your progress. The local tests check the examples saved here. A passing example is useful for understanding the fix, but it does not mean every historical submission was rerun.

8. Debugging checklist and references

Check	Question to ask
Clock contract	How many rising edges are required by the protocol?
State memory	Which register remembers where the design is between edges?
NBA old value	Which expressions read pre-edge register values?
Counter meaning	Does count mean current index or completed-bit count?
Bit order	Is the stream LSB-first, and does the shift/index convention match?
Final data bit	Is bit 7 stored before stop-bit validation and done?
Parity extension	Is parity reset once per frame and tested after all 9 bits?
Evidence	Is there a success screenshot, source file, or self-checking simulation?

References

[1] HDLBits, 'Fsm serial'. https://hdlbits.01xz.net/wiki/Fsm_serial
Protocol framing, 8 data-bit clocks, stop-bit recovery, synchronous reset. Accessed 30 August 2026.

[2] HDLBits, 'Fsm serialdata'. https://hdlbits.01xz.net/wiki/Fsm_serialdata
One-bit-at-a-time datapath, parallel byte output, LSB-first ordering. Accessed 30 August 2026.

[3] HDLBits, 'Fsm serialdp'. https://hdlbits.01xz.net/wiki/Fsm_serialdp
Odd parity, ninth serial bit, provided parity accumulator, done gating. Accessed 30 August 2026.

[4] AMD, Vivado Design Suite User Guide: Synthesis (UG901), 'For and Repeat Statements,' 2026.1.
https://docs.amd.com/r/en-US/ug901-vivado-synthesis/For-and-Repeat-Statements
Synthesis use of loops for repetitive and bit-slice structures in always blocks. Accessed 30 August 2026.

[5] AMD, UG901, 'Using Blocking and Non-Blocking Procedural Assignments,' 2026.1.
https://docs.amd.com/r/en-US/ug901-vivado-synthesis/Using-Blocking-and-Non-Blocking-Procedural-Assignments
RHS evaluation at statement execution and deferred variable update. Accessed 30 August 2026.

[6] IEEE Standards Association, IEEE 1800-2023. https://standards.ieee.org/ieee/1800/7743/
Active SystemVerilog language standard defining syntax and semantics. Accessed 30 August 2026.

Prepared from Kapil's pasted discussion. Technical claims were checked against the three HDLBits problem statements, AMD synthesis documentation, the active IEEE standard record, and a local self-checking Icarus Verilog simulation.

9. Follow a complete serial frame

Entries 2 and 4: start, data and stop

While idle, a sampled 0 starts a frame. That edge is the start bit, so it must not also write data bit 0. Capture bit 0 on the next rising edge and bit 7 on the eighth data edge. The following edge checks the stop bit.

Edge	Input meaning	Action
Start	0	Enter DATA; set bit index to 0
Next 8 edges	b0 through b7, LSB first	Capture one bit per edge
Stop	Must be 1	Pulse done if the frame is valid
Following edge	Possible next start bit	Return to start detection without losing a back-to-back frame

What happens when the stop bit is missing?

If the stop sample is 0, suppress done and wait for a sampled 1. Zeros during this recovery period are still part of the malformed frame; do not treat each one as a new start. Once a 1 has restored idle framing, a later sampled 0 may start a new frame.

When can out_byte be read?

Read it while done is high. The eight writes have finished before the stop edge, so the assembled byte is ready when that edge validates the frame. The exercise does not require out_byte to be meaningful while done=0.

The x values on page 8 mark positions that have not yet been captured in that example. With the reset-to-zero code on page 7, those positions would contain zero instead. The bit order and final byte are the same.

Sources: https://hdlbits.01xz.net/wiki/Fsm_serial and https://hdlbits.01xz.net/wiki/Fsm_serialdata

10. Add odd parity at the right edge

Entry 6: one more sample before the stop bit

Odd parity means the XOR of the eight data bits and the parity bit is 1. Reset the parity accumulator at the start boundary, include all eight data samples, and include the parity sample. The stop bit must not affect the value used for this check.

```
// Meaning: parity_total contains the XOR accumulated so far.
// Clear it before data bit 0. During DATA and PARITY:
parity_total <= parity_total ^ in;
// At the following STOP edge, the old value includes parity:
// valid_frame = in && parity_total;
```

The snippet shows the timing rule. In the HDLBits exercise, connect the provided parity module and choose its reset timing to implement that rule. Its accumulator toggles on each sampled 1, so inspect its old output at the stop edge before that edge can include the stop bit.

Stage just completed	What the stored XOR contains
Start	0
Data bit 0	b0
Data bit 7	b0 XOR b1 XOR ... XOR b7
Parity bit	All eight data bits XOR parity
Stop test	Use the stored nine-bit XOR; require stop=1

For byte A6, the data has four ones. A parity bit of 1 makes five, so it is valid. A parity bit of 0 makes four, so done must stay low. In both cases a valid stop bit ends the frame. A zero stop bit uses the framing-error recovery from page 11.

If you check parity_total in the same block activation that captures the parity bit, it still contains only the eight data bits. Either test parity_total ^ in at that edge, or wait until the stop edge and test the stored value. Mixing these two conventions causes a one-bit timing error.

Source: https://hdlbits.01xz.net/wiki/Fsm_serialdp

1. Translate the problem into a clock sequence

The pattern text is not enough; the judge cares about the cycle in which each output is high.

The input is a continuous serial stream. On every rising edge, the FSM samples one bit. Reset must place the machine in the same logical condition as if a 0 had just been seen. From that point, only the current run length of consecutive 1s matters until a 0 terminates the run.

Run ending at this sampled bit	Meaning	Post-edge output
`...0 11111 0`	A 0 follows exactly five consecutive 1s. The inserted 0 is data stuffing and must be discarded.	`disc=1, flag=0, err=0`
`...0 111111 0`	A 0 follows exactly six consecutive 1s. This is the HDLC flag delimiter.	`disc=0, flag=1, err=0`
`...0 1111111`	The seventh consecutive 1 has just been sampled. Every additional 1 remains an error.	`disc=0, flag=0, err=1`

> **Important reading of the leading 0**
>
> You do not need a separate state that remembers an old leading 0 forever. Any 0 resets the run length to zero, and reset is defined to behave as though a 0 was already present. Therefore, 'five 1s followed by 0' is enough to recognize the listed discard pattern in this streaming context.

A run-length reference model

Let `r` be the number of consecutive 1s seen before the current edge, and let `b` be the bit sampled at this edge. The required outputs immediately after the edge are:

```
disc = 1  iff  (b == 0) and (r == 5)
flag = 1  iff  (b == 0) and (r == 6)
err  = 1  iff  (b == 1) and (r >= 6)
```

This oracle is the cleanest way to debug the FSM: it says both what event occurred and when its output must be observable. The terminating 0 belongs to the discard or flag decision. The seventh 1 belongs to the error decision.

2. Moore versus Mealy is an output-timing decision

The transition graph can look almost identical while the observable waveform moves across a clock edge.

Property	Moore output	Mealy output
Equation	`y = g(state)`	`y = g(state, in)`
When it can change	When the registered state changes (normally just after a clock edge).	Whenever either the state or the input changes, including between clock edges.
What stores the event	A state entered after the decisive bit is sampled.	No event state is required; the current input completes the decision combinationally.
Risk in this problem	A naive design needs extra pulse states or it announces too early.	A direct output pulse appears before the decisive bit is clocked into the state register.

> **Why Moore?**
>
> Mealy is not globally wrong. It is a valid FSM architecture. Your first architecture fails this particular interface contract because HDLBits explicitly asks for a Moore machine and expects outputs aligned with the registered, post-edge recognition event.

Why the word reg caused confusion

In Verilog, `reg` means a procedural assignment variable. It does not promise a physical flip-flop. Hardware inference comes from where and how the variable is assigned. Your `discReg`, `flagReg`, and `errReg` are assigned in `always @(*)`, so they are combinational signals. Their names end in 'Reg', but no clock stores them.

```
always @(*) begin
    discReg = 0;
    flagReg = 0;
    errReg  = 0;
    ...
    discS: if (!in) discReg = 1;
end

assign disc = discReg;
```

The defaults are good combinational coding style because they prevent latches. But they also make the intended behavior explicit: whenever `state` or `in` changes, the outputs are recalculated immediately.

3. Deep dive: why the first Mealy-style version fails

It tracks the run length correctly. The decisive bug is that disc and flag live before the sampling edge instead of after it.

What the first states actually mean

State	Meaning after the previous edge	Output decision still needed
`s0`	Zero consecutive 1s	None
`s1 ... s4`	One through four consecutive 1s	Keep counting
`discS`	Exactly five consecutive 1s	Need the next bit: 0 means discard, 1 means continue
`flagg`	Exactly six consecutive 1s	Need the next bit: 0 means flag, 1 means error
`error`	Seven or more consecutive 1s	Remain in error while 1s continue

This state interpretation is sound. In fact, it is exactly why the Mealy formulation feels natural: when the machine is in the FIVE state and the live input becomes 0, the pair `(FIVE, 0)` completely identifies a discard. Likewise, `(SIX, 0)` identifies a flag.

The wrong side of the edge

Rising-edge view for the decisive suffix 111110

	edge 1	edge 2	edge 3	edge 4	edge 5	edge 6	edge 7
sampled bit	1	1	1	1	1	0	next
state after edge	ONE	TWO	THREE	FOUR	FIVE	DISC	...
Mealy disc	0	0	0	0	0	0	0
Moore disc	0	0	0	0	0	1	0

The Mealy pulse existed before edge 6, not after it.

Figure 1. The accepted Moore output is high in the cycle after edge 6, because edge 6 is where the terminating 0 becomes part of registered history. The direct Mealy output is high only before edge 6 while the old FIVE state and the new, not-yet-sampled 0 coexist.

At the decisive edge, nonblocking assignment updates `state <= next_state`. In the first design, `next_state` is `s0` for `(discS, in=0)`. Immediately after the state update, the combinational block runs again in `s0`, applies `discReg=0`, and erases the pulse. A checker that evaluates the expected registered-cycle output after the edge sees 0 instead of 1.

> **Measured local trace**
>
> For the final 0 of `111110`: first design PRE-edge `state=discS, disc=1`; first design POST-edge `state=s0, disc=0`. Working design PRE-edge `state=discarded, disc=0`; working design POST-edge `state=discardedFinal, disc=1`.

4. The same failure for flag, and the partial success of err

Not all three outputs fail in exactly the same way, which can make the waveform confusing.

Event	First design before decisive edge	First design after edge	Required after edge
six 1s, then 0	flagg + in=0 -> flag=1	s0 -> flag=0	flag=1
seventh 1	flagg + in=1 -> err=0	error + in=1 -> err=1	err=1
0 after error run	error + in=0 -> err=0	s0 -> err=0	err=0

The flag output has the same edge-placement bug as disc. The terminating 0 makes `flagReg` high before the edge, but that edge also moves the FSM to `s0`, so flag is gone after the edge.

The error output is more deceptive. On the seventh 1, the edge moves the FSM into `error` while `in` is still 1, so `errReg` becomes high after the edge. That happens to line up with the required start of error. However, when the next 0 arrives between edges, the Mealy equation drops err before that 0 is sampled. A pure Moore error output remains high through the current error cycle and drops only after the zero edge.

> **Why one correct-looking output does not rescue the architecture**
>
> A design can accidentally have the right post-edge timing on one transition and the wrong timing on others. Classification depends on the output equation, not on whether a few waveform points happen to match.

A scheduling timeline to remember

Moment	What hardware/model does	First Mealy outputs
Between edges	Input changes and combinational logic settles.	May change immediately because they depend on in.
At rising edge	State flip-flops sample next_state; synchronous reset is also sampled.	Old state and current input caused the pre-edge value.
Just after edge	Nonblocking state update completes; combinational logic settles again.	Recomputed from the new state, often clearing disc/flag.

5. Why the output assign became Mealy-style

The output port can be driven by a continuous assignment, but the operands decide whether the behavior is Moore or Mealy.

```
assign disc = (state == discarded) && (in == 1);
```

This is structurally Mealy-style because `disc` is a direct combinational function of both `state` and `in`. The keyword `assign` is not the problem by itself. A Moore output can also use `assign`, provided its expression depends only on registered state.

Expression	Classification	What it means here
`assign disc = (state == FIVE);`	Moore	High after the fifth 1, before the machine knows whether the next bit is 0 or 1. Too early and logically ambiguous.
`assign disc = (state == FIVE) && !in;`	Mealy	Correct logical condition for a live terminating 0, but pulse occurs before that 0 is sampled. Wrong phase for this task.
`assign disc = (state == FIVE) && in;`	Mealy	Wrong polarity for discard. It is high while the stream is extending from five to six 1s, which is the route toward flag/error.
`assign disc = (state == DISC_PULSE);`	Moore	Correct: the pulse state was entered only after a 0 following exactly five 1s was sampled.

> **Fast classification test**
>
> Ignore variable names and ask: Can the output change if only `in` changes while the clock and state stay fixed? If yes, that output is Mealy-style. If no, and it depends only on registered state, it is Moore-style.

6. Why the obvious Moore rewrite also fails

Removing in from the output equation fixes the architecture label but creates an information-timing error unless the state space is expanded.

Suppose `discarded` means exactly five consecutive 1s have been sampled, as it does in your working transition graph. The tempting assignment below is pure Moore:

```
assign disc = (state == discarded);
```

But after the fifth 1, the future is not known. The next bit could be 0, producing a discard, or 1, producing six 1s and possibly a flag or error. A state-only output on the FIVE state must give the same output in both futures. Therefore it cannot correctly announce discard yet.

History already stored	Next sampled bit	Correct interpretation	Why disc on FIVE is wrong
`...011111`	`0`	Discard event	It would be right by coincidence, but one edge too early.
`...011111`	`1`	Now six 1s; wait for 0 (flag) or 1 (error)	It would falsely assert discard even though no stuffed 0 occurred.

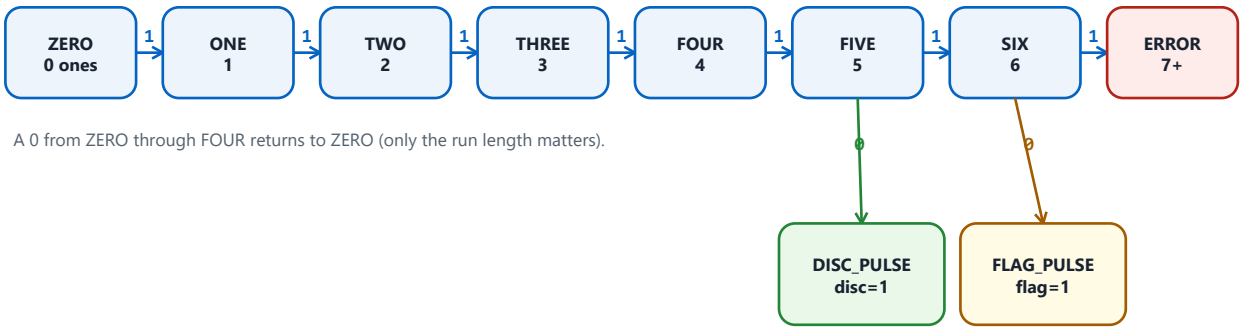
Exactly the same argument applies to `flag = (state == flagged)` when `flagged` means six sampled 1s. After the sixth 1, the next bit is still unknown: 0 completes a flag, while 1 creates an error. Asserting flag in the SIX state would falsely flag every error run on the cycle before its seventh 1.

> **The trap**
>
> If you assert from FIVE/SIX, the output is Moore but too early. If you add the live input to wait for the terminating 0, the logical decision is right but the output becomes Mealy and appears before the edge. The solution is not a cleverer assign expression. The solution is more state memory.

7. Why 10 Moore states are natural

A Moore state must encode enough past information to determine both the next transition and the output for the entire current cycle.

State category	How many	Why distinct
Run length 0 through 6	7	Each length reacts differently to a future 1 or 0.
Error run 7 or more	1	All longer runs can merge because err stays high on 1 and clears on 0.
Discard pulse	1	Same new run length as an ordinary 0, but disc must be 1 for this cycle.
Flag pulse	1	Same new run length as an ordinary 0, but flag must be 1 for this cycle.
Total	10	Requires at least 4 state bits because 3 bits encode only 8 states.



A 0 from ZERO through FOUR returns to ZERO (only the run length matters).

DISC_PULSE / FLAG_PULSE: next 0 -> ZERO; next 1 -> ONE.

SIX + 1 -> ERROR. ERROR + 1 -> ERROR. ERROR + 0 -> ZERO.

Green and yellow states are event memory: they store what was just recognized at the clock edge.

Figure 2. Conceptual state graph. Your names `discarded` and `flagged` are the FIVE and SIX count states. Your names `discardedFinal` and `flaggedFinal` are the actual output pulse states.

Why these states stay separate

After a terminating 0, ZERO, DISC_PULSE, and FLAG_PULSE all have the same run length: zero consecutive 1s. They still cannot be merged in a Moore machine because their current outputs differ. Moore-state minimization may merge states only when their outputs and future behavior are equivalent.

8. Walk through the working Moore FSM

The extra states move recognition across the clock edge and hold each event for exactly one cycle.

Current state meaning	Input sampled	Next registered state	Output after edge
FOUR: four 1s	`1`	FIVE / discarded	All 0; still waiting
FIVE: five 1s	`0`	DISC_PULSE / discardedFinal	`disc=1`
FIVE: five 1s	`1`	SIX / flagged	All 0; still waiting
SIX: six 1s	`0`	FLAG_PULSE / flaggedFinal	`flag=1`
SIX: six 1s	`1`	ERROR / errorFound	`err=1`
ERROR: 7+ 1s	`1`	ERROR / errorFound	`err=1`
ERROR: 7+ 1s	`0`	ZERO / s0	All 0

Why the pulse states inspect the next input

```
discardedFinal: next_state = (in) ? s1 : s0;
flaggedFinal:   next_state = (in) ? s1 : s0;
```

While `disc` or `flag` is high, the stream does not pause. The next input bit is already being presented for the next rising edge. If that bit is 1, it must become the first 1 of a new run, so the machine goes to `s1`. If it is 0, the run length remains zero, so it goes to `s0`.

A common near-miss is to force every pulse state unconditionally back to `s0`. That produces the right one-cycle output but silently loses an overlapping 1 arriving during that output cycle. Your conditional transitions avoid that bug.

Why these outputs are Moore

```
assign disc = (state == discardedFinal);
assign flag = (state == flaggedFinal);
assign err  = (state == errorFound);
```

Each expression depends only on the registered current state. A change on `in` may change `next_state`, but it cannot directly change any output. The output changes only after a rising edge changes `state`.

9. Cycle-by-cycle traces

These are the traces to reproduce mentally when HDLBits highlights a mismatch.

Discard trace: reset/previous 0, then 1111101

Sample	Bit	State before edge	State after edge	disc after edge
1	1	ZERO	ONE	0
2	1	ONE	TWO	0
3	1	TWO	THREE	0
4	1	THREE	FOUR	0
5	1	FOUR	FIVE	0
6	0	FIVE	DISC_PULSE	1
7	1	DISC_PULSE	ONE	0

Flag trace: reset/previous 0, then 11111101

Sample	Bit	State before edge	State after edge	flag / err
1-5	11111	ZERO through FOUR	FIVE	0 / 0
6	1	FIVE	SIX	0 / 0
7	0	SIX	FLAG_PULSE	1 / 0
next	1	FLAG_PULSE	ONE	0 / 0

Error trace: reset/previous 0, then 111111110

Sample	Bit	State before edge	State after edge	err after edge
1-6	111111	ZERO through FIVE	SIX	0
7	1	SIX	ERROR	1
8	1	ERROR	ERROR	1
9	0	ERROR	ZERO	0

10. Read the Verilog using event regions

Follow the evaluation order below to see when the output changes.

Step	Code involved	Meaning
1. Before edge	`always @(*)`	Current state and current input settle next_state. A Mealy output may also settle now.
2. Rising edge	`always @(posedge clk)`	The state register schedules state <= next_state using nonblocking assignment.
3. NBA update	`state <= next_state`	The registered current state takes its new value after the edge-triggered block evaluates.
4. Re-evaluation	`always @(*) / assign`	Anything depending on state recalculates. Moore outputs now describe the bit just sampled.

In the failing design, the disc and flag decisions exist during step 1, then disappear during step 4 because state has already returned to `s0`. In the working design, step 4 sees `discardedFinal` or `flaggedFinal`, so the output begins after the decisive edge and remains stable for the cycle.

Synchronous reset consequence

Because reset is synchronous, `state` changes to `s0` only at a rising edge where `reset=1`. The combinational output equations then become zero because none of the output states is active. The first attempt's declaration initializers on `discReg`, `flagReg`, and `errReg` are not a substitute for resetting state and are unnecessary because the combinational block assigns defaults on every path.

Could a registered Mealy output pass?

A designer could sample a Mealy condition into output flip-flops at the rising edge. Externally, that can place the pulse in the required post-edge cycle. But those output flip-flops themselves become state memory. If the complete machine state includes them, the external behavior is Moore-like. For this exercise, the explicit pulse states are clearer, match the hint, and make the timing visible in the state graph.

> **Do not memorize 'continuous assign equals Mealy'**
>
> Continuous assignment is only a combinational mechanism. `assign y = (state == S);` is Moore-style. `assign y = (state == S) && in;` is Mealy-style. The dependency graph, not the syntax keyword, determines the classification.

11. Code review of the working solution

The architecture is correct. A few naming and style changes can make the reasoning easier to see without changing behavior.

Observation	Assessment	Recommendation
4-bit state register	Correct for 10 states.	Keep it, or use SystemVerilog enum logic [3:0].
`next_state = state` before case	Good defensive default; prevents a latch if a branch is missed.	Keep the default and the default case.
`discarded` / `flagged`	Functionally correct, but names sound like outputs are already active.	Rename conceptually to FIVE and SIX.
`discardedFinal` / `flaggedFinal`	These are the true one-cycle output states.	Names DISC_PULSE and FLAG_PULSE reveal timing better.
Blocking assignments in combinational logic	Correct.	Continue using = for next-state logic and <= for the clocked state register.
Output continuous assignments	Correct Moore decoding because they depend only on state.	Keep them simple and state-only.

> **Clearer state names**
>
> Read `discarded` as FIVE and `flagged` as SIX. Then read `discardedFinal` as DISC_PULSE and `flaggedFinal` as FLAG_PULSE. The entire design becomes almost self-explanatory.

How to work through a similar FSM

- Write each required output as a statement about the bit sampled at the current edge and the history before that edge.
- Give every state a sentence meaning such as 'exactly five consecutive 1s have been sampled'.
- If a Moore output needs to remember that an event just happened, create a one-cycle event state.
- While in an event state, still consume the next stream bit; do not assume the stream pauses.
- Trace the last two edges around every output transition: the edge that begins the pulse and the edge that ends it.

12. Verification evidence and debugging checklist

A small reference model separates pattern-recognition bugs from state-encoding and scheduling bugs.

Both source files were compiled with Icarus Verilog in SystemVerilog 2012 mode. The working Moore version was compared after each rising edge against the run-length oracle from Section 1. Directed discard, flag, and error cases were followed by deterministic pseudo-random input.

Check	Result
Working source syntax/elaboration	PASS
Directed 111110 discard event	PASS: disc high after terminating-zero edge
Directed 1111110 flag event	PASS: flag high after terminating-zero edge
Directed 7+ ones error event	PASS: err high from seventh sampled 1
Reference-model comparison	PASS: 1022 post-edge checks, zero mismatches

When HDLBits shows a red mismatch

Question	What it diagnoses
Is the pulse one cycle early or late?	Moore/Mealy placement or an extra/missing state.
Does it occur before the edge but disappear after it?	Direct input dependence in a Mealy output.
Does disc assert on a future flag/error run?	Output decoded from the FIVE state too early.
Does flag assert before the seventh 1 error?	Output decoded from the SIX state too early.
Is the first 1 after a pulse lost?	Pulse state unconditionally returned to ZERO instead of inspecting in.
Does err flicker when input changes before the edge?	Combinational err dependence on in.

> **Final diagnosis**
>
> Your first FSM had enough memory to count the 1s, but not enough Moore-state memory to hold the completed discard and flag events after their terminating 0 was sampled. The working FSM adds exactly that memory, one state per distinct output event.

A. The decisive blocks from the first attempt

These excerpts show why the design is Mealy-style even though it uses reg variables.

```
always @(*) begin
    next_state = state;
    discReg = 0;
    flagReg = 0;
    errReg = 0;

    case (state)
        ...
        discS: begin
            if (!in) begin
                discReg = 1;
                next_state = s0;
            end else begin
                next_state = flagg;
            end
        end

        flagg: begin
            if (!in) begin
                flagReg = 1;
                next_state = s0;
            end else begin
                next_state = error;
            end
        end

        error: begin
            if (in) begin
                errReg = 1;
                next_state = error;
            end else begin
                errReg = 0;
                next_state = s0;
            end
        end
    endcase
end
```

The critical lines are not the transitions; those correctly count the run. The critical lines are the output assignments inside branches that test `in`. They create direct input-to-output combinational paths.

Also notice that disc and flag are asserted in the same combinational evaluation that selects `s0` as next state. At the rising edge, the state transition that consumes the 0 simultaneously removes the condition responsible for the pulse. That is the exact reason the pulse does not survive into the expected post-edge cycle.

B. Working Moore source - part 1

Saved separately as fsm_hdlc_working_moore.v; reproduced here so the explanation remains self-contained.

```
// Working HDLBits solution for: https://hdlbits.01xz.net/wiki/Fsm_hdlc
// Ten-state Moore FSM: outputs depend only on the registered current state.

module top_module (
    input  clk,
    input  reset,    // Synchronous reset
    input  in,
    output disc,
    output flag,
    output err
);

    reg [3:0] state, next_state;

    parameter s0             = 4'b0000;
    parameter s1             = 4'b0001;
    parameter s2             = 4'b0010;
    parameter s3             = 4'b0011;
    parameter s4             = 4'b0100;
    parameter discarded      = 4'b0101;
    parameter flagged        = 4'b0110;
    parameter errorFound     = 4'b0111;
    parameter discardedFinal = 4'b1000;
    parameter flaggedFinal   = 4'b1001;

    always @(posedge clk) begin
        if (reset)
            state <= s0;
        else
            state <= next_state;
    end

    always @(*) begin
        next_state = state;
```

B. Working Moore source - part 2

```
        case (state)
            s0:             next_state = (in) ? s1         : s0;             // 0 ones
            s1:             next_state = (in) ? s2         : s0;             // 1 one
            s2:             next_state = (in) ? s3         : s0;             // 2 ones
            s3:             next_state = (in) ? s4         : s0;             // 3 ones
            s4:             next_state = (in) ? discarded  : s0;             // 4 ones
            discarded:      next_state = (in) ? flagged    : discardedFinal; // 5 ones
            discardedFinal: next_state = (in) ? s1         : s0;             // disc pulse
            flagged:        next_state = (in) ? errorFound : flaggedFinal;   // 6 ones
            flaggedFinal:   next_state = (in) ? s1         : s0;             // flag pulse
            errorFound:     next_state = (in) ? errorFound : s0;             // 7+ ones
            default:        next_state = s0;
        endcase
    end

    assign disc = (state == discardedFinal);
    assign flag = (state == flaggedFinal);
    assign err  = (state == errorFound);

endmodule
```

What made this version work

The transition into `discardedFinal` or `flaggedFinal` happens at the same rising edge that samples the terminating 0. The state-only output decode then asserts for the entire following cycle. No live input is present in the output equations.

Reference

HDLBits, 'Fsm hdlc': https://hdlbits.01xz.net/wiki/Fsm_hdlc (accessed 29 August 2026). The page specifies the three stream patterns, synchronous-reset behavior, and a Moore state machine of about 10 states.

Prepared from Kapil's original failing Mealy-style source and working 10-state Moore source. Local simulation evidence was generated specifically for this document.